



Compilers

Guillermo A. Pérez

Lecture Y: Good code generation

TL;DR: This lecture in short

How do I generate better code?

By now, we can compile source programs into target programs. However, a basic CS question remains: **is it optimal?**

Main references

- **Introduction to Language Theory and Compilers.** Gilles Geeraerts and G. A. Pérez. 2019.
- **Crafting a Compiler.** Charles N. Fischer et al. 2010. Pearson Education.
- **Compiler Design.** Helmut Seidl et al. 2012. Springer
- **Compilers: Principles, Techniques, Tools.** Alfred Aho et al. 2006. Pearson Education.
- Online: Wikipedia

Required and target competences

What tools do we need?

Graph and tree algorithms (data structures and graph algorithms)

What skills will we obtain?

- theory: the limits of optimal-code generation
- practice: code-generation data structures, algorithms, and heuristics

How will these skills be useful?

- Besides their relevance to the project, these will allow you to understand what gcc and other compilers are doing with your precious code.

Outline

1. Cost model
2. Good code generation (local view)
3. Peephole optimizations
4. Expression evaluation
5. Other optimizations

How do we compare generated code?

Program and instruction costs

We associate a **cost** with compiling and running a program. Common cost measures are functions of

- the size of the generated program,
- its power consumption, and
- its running time.

Cost model

A simple cost recipe

We will assume every instruction costs 1 unit plus the cost associated with addressing modes of the operands.

- Operations on registers have additional cost 0. E.g. `addi $t0, $t1, $a2`
- Those with constants or memory locations have additional cost 1. E.g. `addi $t0, 5, $a2`
- Those with memory locations and offsets have additional cost 2. E.g. `lw $t2, 20($a1)`

Good-code generation

Minimal-cost programs

We are interested in generating the target program with **minimal cost** for the input program. The cost of the program is the sum of the costs of its instructions.

Good code

To generate good code w.r.t. our cost model, we will use

- basic blocks,
- control-flow graph,
- **next-use information** for variables,
- and will first focus on **locally-good code**.

Next-use information

Next usage

One of the most basic pieces of information we can use for optimizations:

- when will (the contents of the) variable x next be used?

Definition (Usage)

Let $i < j$. If the i -th instruction assigns to x some value, the control can flow from there to the j -th instruction and x is an operand in the j -th instruction, then **the value of x computed at i is used at j** .

Definition (Liveness)

Let $i < k$. If the value of x computed at i is used at k then **x is live at all $i \leq j < k$** .

Usage and liveness information

Data structure

The usage and liveness information for variables should be associated to individual instructions.

- For the algorithm below, temporary information can be kept as **additional information** in the symbol table.
- Alternatively, we can keep a table for the information per basic block.

Algorithm

We go through the instructions in the block from last to first. For every instruction $i : x := y \text{ op } z$ we do the following.

1. Attach to instruction i the usage and liveness information of x, y, z .
2. Set x to **not live** and **no next use**.
3. Set y, z to **live** and the next uses of y, z to i .

Example

```
1   y := 1 + 2
2   x := 5
3   w := x + y
4   z := x - w
```

Instructions

Label the instructions with a set L of live variables and a function U from variables to their next usage.

Data flow in a basic block

Local optimization

We will construct, via an in-order pass, a structure representing how information flows between variables.

Basic block directed acyclic graph G

1. There is a node in G for each initial value of the variables in the block.
2. There is a node N associated to each instruction in the block. Its children are the nodes corresponding to instructions where the operands of i were last defined. Nodes with the same children are “merged”.
3. A node is labelled by the operator applied at its corresponding instructions and with the list of variables for which it is the last definition within the block.
4. Nodes with values that can be used by successor blocks are marked as live on exit. E.g. return instructions.

Example: Local common sub-expressions

Example

- Construct the DAG for the basic block from the previous exercise.
- Construct the DAG for the following.

```
1  a := b + c
2  b := a - d
3  c := b + c
4  d := a - d
```

- Assume initial definitions a_0, b_0, c_0, d_0 .
- How many nodes are labelled by more than one variable as being definitions for them?

Using the DAG of a basic block

Code-improving transformations

- **Eliminating local common sub-expressions:** instructions that compute a value that has already been computed.
- **Dead-code elimination:** instructions that compute a value that is never used
- **Instruction reordering:** to reduce the time a temporary value is kept in a register
- **Reorder operands of instruction:** using algebraic laws, sometimes simplifying expression evaluation

Example: Dead-code elimination

Example

Construct the DAG for:

```
1   a := b + c
2   b := b - d
3   c := c + d
4   e := b + c
5   return a * b
```

- All roots in the DAG which are not labelled with live variables can be iteratively removed.
- What does the resulting DAG look like?

Simple, not-too-bad code-generation

Data structure

Since registers are limited, we can keep track of the following to minimize store/load sequences.

1. **Register descriptors:** variable names whose current value is in a register (per register).
2. **Address descriptors:** all locations where the current value of a variable is stored.

Algorithm

For every instruction $i : x = y \text{ op } z$ we do the following.

1. Select registers R_x, R_y, R_z for the variables using $\text{getReg}(x := y \text{ op } z)$.
2. If y is not loaded in R_y then issue the instruction to do so.
3. If z is not loaded in R_z then issue the instruction to do so.
4. Issue the instruction $R_x := R_y \text{ op } R_z$.

Notes on the algorithm

Copy instructions

Note that copy instructions $x := y$ result in **no machine-instruction output** if $R_x = R_y$.

End of block

For variables which are marked as live on exit, we issue a store instruction if the value of the variable is currently only in a register.

Descriptor management

We update information about the descriptors following the semantics of machine instructions we are issuing.

Register allocation (1/2)

Registers for operands

R_y given by $\text{getReg}(x := y \text{ op } z)$ can be chosen as follows.

1. If y is currently in a register r then $R_y = r$.
2. If y is not in a register but the register r is currently empty then $R_y = r$.
3. The remaining case is the difficult one. Let r be a candidate register:
 - 3.1 If all variables whose descriptor says their value is in r also have another location then return r .
 - 3.2 If the only variable whose descriptor says their value is in r is x and $x \notin \{y, z\}$ then return r . (We will overwrite it anyways!)
 - 3.3 If all variables ... are not used after the instruction (i.e. they are not live), again return r .
 - 3.4 Otherwise **spill** the variables into memory.

Register allocation (2/2)

Registers for result

The differences with the choice of register for operands are:

- Since the value of x is being computed then a register r currently holding the value of x is always a safe choice.
- If y or z are not live after the current instruction, then R_y or R_z can be safely used.

Break

Coffee? 5-min plank?

Peephole optimizations

A sliding window or peephole

A simple but effective technique consists in examining a **sliding window** over target instructions. These can be replaced by a shorter or faster sequence of instructions whenever possible

Common optimizations

- Redundant-instruction elimination
- Flow-of-control optimizations
- Algebraic simplifications
- Use of machine idioms (e.g. auto-increments for addresses)

Redundant instructions: load and store

Naively-generated code

A simple way to generate the target program without caring about the limited amount of registers is to load and store all computed values.

```
1      lw $t0, ($t2)
2      sw $t0, ($t2)
```

Eliminating some load-store sequences

Consider a peephole with a load-and-store sequence within a basic block.

- The store instruction can be removed if it has no label (i.e. it is not the target of a jump).

Unreachable code elimination

Constant-condition jumps

Debug information is sometimes removed from production code as follows.

```
1      if 0 != q goto L2
2      print debugging information
3  L2:
```

Unreachable code elimination

- The condition of the jump in the code above always evaluates to true!
- Instead of a conditional jump, it can be replaced by an unconditional jump and the debugging code becomes unreachable.
- The debugging code can then be removed.

Flow-of-control optimizations

Conditionals and jumps

Conditional statements are translated into jumps in low-level languages. We can end up with sequences like the following.

```
1      goto L1
2      ...
3  L1:  goto L2
```

Inlining double jumps

The first jump can be re-targeted to L2. If there are no other jumps to L1 then it can be removed (if it is preceded by an unconditional jump!)

- Note that inlining can also be done if the first jump were conditional!

Algebraic simplification

A finer cost model

For this optimization, we first need to consider the cost of arithmetic operations.

- Squaring (x^2), if exponentiation is implemented in the ALU, is usually more expensive than multiplication $x \times x$.
- Multiplication and division by a power of 2 is usually cheaper to achieve via shifts.
- Division by a constant can be achieved via multiplication with a **magic number** (see multiplicative inverse in modular arithmetic).

Identities

The following instructions can be eliminated because of algebraic identities:

- $x = x + 0$
- $x = x * 1$

Global allocation of registers

From basic blocks to programs

In our simple code-generation algorithm, we stored final register values (corresponding to live-on-exit variables) into memory. If those values are used immediately in subsequent blocks

- Why do we not keep them in a register?

Example: Loops

- Most of the “action” in programs happens inside loops.
- We can focus on keeping variable values used in loops in registers.
- Then, proceed to containing loops and try to do the same.
- (Remember we have a limited number of registers!)

Systematic spilling via graph colouring

Two-pass algorithm

- First, assume you have an infinite number of registers and generate code using them.
- Then, construct a **register-interference graph**. A k -colouring of the graph gives a register allocation to execute the block without spilling.
- Chaitin 1982: several heuristics for colouring the graph using a small number of colours exist.

Interference graph

1. Nodes correspond to the (infinitely-many) registers used in the generated code (i.e. the variables).
2. An edge connects two nodes if they are simultaneously live (i.e. they are both live at some instruction).

Example

```
1   y := 1 + 2
2   x := 5
3   w := x + y
4   z := x - w
```

Instructions

- Construct the interference graph for the code above

Optimal register usage for expressions

Ershov Number (a.k.a. Strahler number)

Let us label a binary expression tree as follows.

- Leaves are labelled with 1.
- Inner nodes with one child have the same label as their child.
- Inner nodes with two children have the maximal label amongst its children's labels if they are different; $1 +$ the label of its children if they are the same.

Theorem

The Ershov Number of the root corresponds exactly to the minimal number of registers required to evaluate the expression without using additional memory nor algebraic properties of the expression operators.

Example: evaluating an expression tree

Expression tree

Build the binary expression tree for t4 in:

```
1   t1 := a - b
2   t2 := c + d
3   t3 := e * t2
4   t4 := t1 + t3
```

Ershov Number

Label the tree with its Ershov Numbers.

- Can you evaluate the sub-expression corresponding to every node with the corresponding optimal number of registers?

The Sethi-Ullman algorithm (1/2)

Consider registers $R_b, R_{b+1}, \dots, R_{b+k-1}$ for k the Ershov Number of the given node and $b \geq 1$ a **base number**. We will leave the result of the computation in register R_{b+k-1} .

Algorithm for equal-children nodes (labelled $k - 1$)

1. Recursively generate code for the right child using base b . The result of the right child appears in R_{b+k-1} .
2. Recursively generate code for the left child using base $b - 1$; its result appears in R_{b+k-2} .
3. Generate the instruction $R_{b+k-1} = R_{b+k-2} \text{ op } R_{b+k-1}$.

The Sethi-Ullman algorithm (2/2)

Different-children nodes (labelled $m < k$)

1. Recursively generate code for the **big child**, using base b ; the result appears in register R_{b+k-1} .
2. Recursively generate code for the **small child**, using base b ; the result appears in register R_{b+m-1} . (Note $R_{b+m}, \dots, R_{b+k-1}$ are not used in the computation!)
3. Generate the instruction $R_{b+k-1} = R_{b+m-1} \text{ op } R_{b+k-1}$ (or vice versa if the right child is the small child).

Exercise

Instructions

- Apply the Sethi-Ullman algorithm to the previous expression-tree example (slide 29).

Expression evaluation with spilling

Modifications to the Sethi-Ullman algorithm

If the input node has label $k > r$ with r the number of registers:

- Pick a “big child” with label at least r .
- Recursively generate code for the big child using $b = 1$. The result will appear in register R_r .
- Store the value of R_r in memory.
- Generate code for the little child: If it has a label of at least r then use base $b = 1$. If the label is $j < r$ then pick $b = r - j$. Recursively apply this algorithm to the little child and the result appears in R_r .
- Load into R_{r-1} the value computed for the big child.
- Issue the instruction to compute into R_r the value of the node.

Data-flow-based optimizations

From basic blocks to control-flow graphs

- We have covered **local optimizations** — within basic blocks.
- These can be extended to global optimizations (more intricate).
- **Data-flow analyses** consist in deriving information about the flow of data along all possible execution paths of a program.

Possible optimizations

- Global common sub-expression elimination
- Copy propagation (eliminate the copy and inline the copied value)
- Code motion: (invariant) code from inside a loop to before the loop
- Reaching definitions: detect if a variable is used before initialization
- Live variables: detect if a variable is live in a block
- Available expressions: detect recomputation of an expression with variable values not having changed

Machine-instruction-level parallelism

Pipelining

Another family of optimization exploits the fact that modern CPUs **pipeline instructions**. A classic RISC pipeline consists of:

1. Instruction fetch
2. Instruction decode and register fetch
3. Execute
4. Memory access
5. Register write back

Hardware pipelining

- Instruction scheduling can be hardware or software-based.
- Good compilers generate code with parallelization opportunities. (Reordering instructions to optimize register usage may reduce parallelization opportunities!)